Algoritmos y Estructuras de Datos

Trabajo Final

Integrantes del grupo

- Nombre y Apellido: Ulises Guerra
- Nombre y Apellido: Maximo Toledo

1- Introducción al trabajo práctico

En el siguiente informe se expondrá el conocimiento de la cátedra sobre implementación de soluciones algorítmicas no lineales, específicamente árboles binarios, aplicado en práctica al codificar los métodos a continuación mostrados. En el ámbito del desarrollo de software moderno, y en particular dentro de las arquitecturas de comercio electrónico (e-commerce), la representación jerárquica de la información resulta indispensable para el modelado de catálogos complejos, la gestión de categorías entrelazadas y la organización de productos finales.
El objetivo principal de este trabajo práctico consiste en resolver desafíos lógicos de navegación, validación e inserción de datos sobre estas estructuras utilizando el lenguaje C#. Para ello, se abordaron tres problemáticas clave del negocio digital:

- La verificación de la simetría y completitud estructural de la jerarquía (EsArbolLleno).
- La generación controlada de rutas y la prevención de colisiones de datos al expandir el catálogo (Agregar).
- La indexación técnica y recuperación de caminos unívocos para la navegación web de cara al usuario (GetUrlSeoPorId y GetURLsSEO).

A través de la aplicación de conceptos como la recursividad, el recorrido en profundidad (DFS) y el procesamiento secuencial de rutas, este proyecto busca demostrar cómo las estructuras de datos avanzadas optimizan el manejo de información jerárquica, garantizando la escalabilidad del sistema y la integridad de los datos en escenarios del mundo real.

2- Código fuente de los métodos implementados

2.1- GetUrlSeoPorId:

```
        public string GetUrlSeoPorId(ArbolGeneral<ItemCat> arbol, int id){
            if(arbol == null) {
```

```
                return "";
            }
            string urlResultado = GetUrlSeoPorIdAuxiliar(arbol, id, "");
            return urlResultado ?? "";
        }
        private string GetUrlSeoPorIdAuxiliar(ArbolGeneral<ItemCat> nodo, int
id, string urlAcumulado){
            if(nodo == null){
                return null;
            }
            string nuevaUrl = urlAcumulado + "/" +
nodo.getDatoRaiz().Nombre.ToLower().Replace(" ", "-");
            if(nodo.esHoja() && nodo.getDatoRaiz().Id == id){
                return nuevaUrl;
            }
            if(nodo.getHijos() != null){
                foreach(ArbolGeneral<ItemCat> nodoHijo in nodo.getHijos()){
                    string urlHijo = GetUrlSeoPorIdAuxiliar(nodoHijo, id,
nuevaUrl);
                    if(!string.IsNullOrEmpty(urlHijo)){
                        return urlHijo;
                    }
                }
            }
            return null;
        }
```

2.2 GetUrlSeoPorId:

```
        public List<string> GetURLsSEO(ArbolGeneral<ItemCat> arbol)
        {
            List<string> urls = new List<string>();
            if(arbol == null) return null;
            GetURLsSEOAuxiliar(arbol, "", urls);
            return urls;

        }

        private void GetURLsSEOAuxiliar(ArbolGeneral<ItemCat> nodo, string
urlAcumulado, List<string> urls)
        {
            if(nodo == null) return;
            string nuevaUrl = urlAcumulado + "/" +
nodo.getDatoRaiz().Nombre.ToLower().Replace(" ", "-");
            if (nodo.esHoja())
            {
                urls.Add(nuevaUrl);
            }
            if(nodo.getHijos() != null){
                foreach(ArbolGeneral<ItemCat> hijo in nodo.getHijos())
                {
                    GetURLsSEOAuxiliar(hijo, nuevaUrl, urls);
                }
            }
        }
```

2.3 ConsultaNiveles:

```
        public List<List<string>> ConsultaNiveles(ArbolGeneral<ItemCat> arbol){
            List<List<string>> listaFinal = new List<List<string>>();
            if(arbol == null){
                return listaFinal;
```

```
            }
            List<ArbolGeneral<ItemCat>> nodosNivel = new
List<ArbolGeneral<ItemCat>>();
            Queue<ArbolGeneral<ItemCat>> cola = new
Queue<ArbolGeneral<ItemCat>>();
            cola.Enqueue(arbol);
            while(cola.Count > 0){
                int nodosEnEsteNivel = cola.Count;
                List<string> nombresNivel = new List<string>();
                for (int i = 0; i < nodosEnEsteNivel; i++){
                    ArbolGeneral<ItemCat> nodoActual = cola.Dequeue();
                    nombresNivel.Add(nodoActual.getDatoRaiz().Nombre);
                    if(nodoActual.getHijos() != null){
                        foreach(ArbolGeneral<ItemCat> hijo in
nodoActual.getHijos()){
                            cola.Enqueue(hijo);
                        }
                    }
                }
                listaFinal.Add(nombresNivel);
            }
            return listaFinal;
        }
```

2.4 Agregar:

```
       public void Agregar(ArbolGeneral<ItemCat> arbol, ItemCat dato, string
rutaAlPadre){
            if(arbol == null || dato == null){
                return;
            }
            if(string.IsNullOrWhiteSpace(rutaAlPadre)){
                throw new ArgumentException("Un producto debe tener una ruta
de categorias validas");
            }
            ArbolGeneral<ItemCat> nodoActual = arbol;
            string[] pasos = rutaAlPadre.Split('/',
StringSplitOption.RemoveEmptyEntries);
            foreach(string paso in pasos){
                ArbolGeneral<ItemCat> hijoEncontrado = null;
                foreach (ArbolGeneral<ItemCat> hijo in
nodoActual.getHijos()){
                    if(hijo.getDatoRaiz().Nombre.Equals(paso,
StringComparison.OrdinalIgnoreCase)){
                        hijoEncontrado = hijo;
                        break;
                    }
                }
                if(hijoEncontrado != null){
                    nodoActual = hijoEncontrado;
                }
                else{
                ItemCat categoria = new ItemCat(paso,
TipoElemento.Categoria);
                ArbolGeneral<ItemCat> arbolN = new ArbolGeneral<ItemCat>
(categoria);
                nodoActual.agregarHijo(arbolN);
                nodoActual = arbolN;
                }
            }
```

```
            bool productoDuplicado = false;
            foreach(ArbolGeneral<ItemCat> hijo in nodoActual.getHijos()){
                if(hijo.esHoja() &&
hijo.getDatoRaiz().Nombre.Equals(dato.Nombre,
StringComparison.OrdinalIgnoreCase)){
                    productoDuplicado = true;
                    break;
                }
            }
            if(productoDuplicado == true){
                throw new InvalidOperationException($"El producto
'{dato.Nombre}' ya existe en esta categoria");
            }
            ArbolGeneral<ItemCat> hijoNuevo = new
ArbolGeneral<ItemCat>(dato);
            nodoActual.agregarHijo(hijoNuevo);
        }
```

2.5 Buscar:

```
        public List<ItemCat> Buscar(ArbolGeneral<ItemCat> arbol, string
elementoABuscar)
        {
            List<ItemCat> resultadoAcumulado = new List<ItemCat>();
            if(arbol == null || string.IsNullOrWhiteSpace(elementoABuscar))
return resultadoAcumulado;
            BuscarAuxiliar(arbol, elementoABuscar, resultadoAcumulado);
            return resultadoAcumulado;
        }
        private void BuscarAuxiliar(ArbolGeneral<ItemCat> nodo, string
elementoABuscar, List<ItemCat> listaCompartida){
            if(nodo == null) return;
            if(nodo.getDatoRaiz().Nombre.Contains(elementoABuscar,
StringComparison.OrdinalIgnoreCase)){
                listaCompartida.Add(nodo.getDatoRaiz());
            }
            foreach(ArbolGeneral<ItemCat> hijo in nodo.getHijos()){
                if(hijo != null){
                    BuscarAuxiliar(hijo, elementoABuscar, listaCompartida);
                }
            }

        }
```

2.6 Todos:

```
        public List<ItemCat> Todos(ArbolGeneral<ItemCat> arbol){
            List<ItemCat> resultado = new List<ItemCat>();
            Cola<ArbolGeneral<ItemCat>> pendientes = new
Cola<ArbolGeneral<ItemCat>>();
            if(arbol == null){
                return resultado;
            }
            pendientes.encolar(arbol);
            while (!pendientes.esVacia()){
                ArbolGeneral<ItemCat> actual = pendientes.desencolar();
                if(actual.getDatoRaiz().Tipo == TipoElemento.Producto){
                    resultado.Add(actual.getDatoRaiz());
                }
                foreach(ArbolGeneral<ItemCat> hijo in actual.getHijos()){
                    pendientes.encolar(hijo);
```

```
                }
            }
            return resultado;
        }
    }
```

3-Ideas de mejoras para el programa

- Mejorar la interfaz: el código como API funciona muy bien aun sin las implementaciones hechas por el grupo, sin embargo la interfaz que presenta no satisface adecuadamente, ya que es muy poco intuitiva.

- Manual de uso: en caso de no poder cambiar la interfaz gráfica por razones de fuerza mayor llegamos a la conclusión de que se debería incluir un manual para que el usuario pueda navegar cómodamente por la página.

4-Conclusión y Experiencia:

Durante el desarrollo del proyecto logramos como grupo aumentar nuestro conocimiento y mejorar nuestro manejo sobre la herramienta GitHub. Además consolidamos con práctica los conceptos vistos en la cátedra, más específicamente el tema de árboles generales. Los temas claves más utilizados dentro del trabajo fueron los métodos de búsquedas y cómo cumplen con sus recorridos por un árbol N-ario como DFS y BFS, siendo lo que nos tocó investigar ya que se planteó la posibilidad de la búsqueda dicotómica, tipo de búsqueda específica y exclusiva de los árboles binarios. Al final del proyecto nos encontramos con que la implementación de la recursividad en árboles generales resultó ser un desafío lógico interesante, especialmente al diferenciar el flujo y las reglas de negocio entre nodos internos (categorías) y nodos hoja (productos). Así mismo, el proceso de depuración nos ayudó a comprender la importancia de los tipos de retorno en C# y el manejo estricto de estructuras que pueden contener valores nulos en escenarios del mundo real como los sistemas de comercio electrónico. Por otra parte, la implementación del método Agregar aportó un valor arquitectónico crucial al proyecto. A través de este método, logramos desarrollar un algoritmo de inserción iterativo guiado por ruta, el cual procesa una cadena secuencial (mediante el desglose de un path con Split) para navegar o construir dinámicamente el árbol. Destacamos dos aspectos de esta solución:

1. Creación Autónoma de Infraestructura: El algoritmo actúa con tolerancia a fallos de estructura; si una categoría intermedia de la ruta no existe, el sistema la inicializa en tiempo de ejecución, garantizando que el árbol crezca de forma integrada.

2. Validación de Integridad de Datos: Mediante una búsqueda local exhaustiva en el nodo de

destino, el método previene la duplicidad de productos homónimos, protegiendo la consistencia de la información antes de consolidar la inserción de la nueva hoja.